Python Garbage Collection

Python is one of the most popular programming languages, and its usage is only accelerating. It was named the TIOBE language of the year in 2018 due to its growth rate. Python's ease of use and large community have made it a popular fit for data analysis, web applications, and task automation.

In this post, we'll cover the details of garbage collection in Python. First, we'll review the basics about memory management and why garbage collection is needed. Then we'll look at how Python implements garbage collection. Finally, we'll take a practical look by asking how you should think about garbage collection when writing your Python applications.

What is garbage collection and why do we need It?

If Python is your first programming language, the whole idea of garbage collection might be foreign to you. Let's start with the basics.

Memory management

A programming language uses objects in its programs to perform operations. Objects include simple variables, like strings, integers, or booleans. They also include more complex data structures like lists, hashes, or classes.

The values of your program's objects are stored in memory for quick access. In many programming languages, a variable in your program code is simply a pointer to the address of the object in memory. When a variable is used in a program, the process will read the value from memory and operate on it.

In early programming languages, developers were responsible for all memory management in their programs. This meant before creating a list or an object, you first needed to allocate the memory for your variable. After you were done with your variable, you then needed to deallocate it to "free" that memory for other users.

This led to two problems:

- **Forgetting to free your memory.** If you don't free your memory when you're done using it, it can result in memory leaks. This can lead to your program using too much memory over time. For long-running applications, this can cause serious problems.
- **Freeing your memory too soon.** The second type of problem consists of freeing your memory while it's still in use. This can cause your program to crash if it tries to access a value in memory that doesn't exist, or it can corrupt your data. A variable that refers to memory that has been freed is called a dangling pointer.

These problems were undesirable, and so newer languages added automatic memory management.

Automatic memory management and garbage collection

With automatic memory management, programmers no longer needed to manage memory themselves. Rather, the runtime handled this for them.

There are a few different methods for automatic memory management, but one of the more popular ones uses reference counting. With reference counting, the runtime keeps track of all of the references to an

object. When an object has zero references to it, it's unusable by the program code and thus able to be deleted.

For programmers, automatic memory management adds a number of benefits. It's faster to develop programs without thinking about low-level memory details. Further, it can help avoid costly memory leaks or dangerous dangling pointers.

However, automatic memory management comes at a cost. Your program will need to use additional memory and computation to track all of its references. What's more, many programming languages with automatic memory management use a "stop-the-world" process for garbage collection where all execution stops while the garbage collector looks for and deletes objects to be collected.

With the advances in computer processing from Moore's law and the larger amounts of RAM in newer computers, the benefits of automatic memory management usually outweigh the downsides. Thus, most modern programming languages like Java, Python, and Golang use automatic memory management.

For long-running applications where performance is critical, some languages still have manual memory management. The classic example of this is C++. We also see manual memory management in Objective-C, the language used for macOS and iOS. For newer languages, Rust uses manual memory management.

Now that we know about memory management and garbage collection in general, let's get more specific about how garbage collection works in Python.

How Python implements garbage collection

In this section, we'll cover how garbage collection works in Python.

This section assumes you're using the CPython implementation of Python. CPython is the most widely used implementation. However, there are other implementations of Python, such as PyPy, Jython (Java-based), or IronPython (C#-based).

To see which Python you're using, run the following command in your terminal:

```
python -c 'import platform; print(platform.python_implementation())'
```

There are two aspects to memory management and garbage collection in CPython:

- Reference counting
- Generational garbage collection

Let's explore each of these below.

Reference counting in CPython

The main garbage collection mechanism in CPython is through reference counts. Whenever you create an object in Python, the underlying C object has both a Python type (such as list, dict, or function) and a reference count.

At a very basic level, a Python object's reference count is incremented whenever the object is referenced, and it's decremented when an object is dereferenced. If an object's reference count is 0, the memory for the object is deallocated.

Your program's code can't disable Python's reference counting. This is in contrast to the generational garbage collector discussed below.

Some people claim reference counting is a poor man's garbage collector. It does have some downsides, including an inability to detect cyclic references as discussed below. However, reference counting is nice because you can immediately remove an object when it has no references.

Viewing reference counts in Python

You can use the sys module from the Python standard library to check reference counts for a particular object. There are a few ways to increase the reference count for an object, such as

- Assigning an object to a variable.
- Adding an object to a data structure, such as appending to a list or adding as a property on a class instance.
- Passing the object as an argument to a function.

Let's use a Python REPL and the sys module to see how reference counts are handled.

First, in your terminal, type python to enter into a Python REPL.

Second, import the sys module into your REPL. Then, create a variable and check its reference count:

```
>>> import sys
>>> a = 'my-string'
>>> sys.getrefcount(a)
2
```

Notice that there are two references to our variable a. One is from creating the variable. The second is when we pass the variable a to the sys.getrefcount() function.

If you add the variable to a data structure, such as a list or a dictionary, you'll see the reference count increase:

```
>>> import sys
>>> a = 'my-string'
>>> b = [a] # Make a list with a as an element.
>>> c = { 'key': a } # Create a dictionary with a as one of the values.
>>> sys.getrefcount(a)
4
```

As shown above, the reference count of a increases when added to a list or a dictionary.

In the next section, we'll learn about the generational garbage collector, which is the second tool Python uses for memory management.

Generational garbage collection

In addition to the reference counting strategy for memory management, Python also uses a method called a generational garbage collector.

The easiest way to understand why we need a generational garbage collector is by way of example.

In the previous section, we saw that adding an object to an array or object increased its reference count. But what happens if you add an object to itself?

```
>>> class MyClass(object):
...     pass
...
>>> a = MyClass()
>>> a.obj = a
>>> del a
```

In the example above, we defined a new class. We then created an instance of the class and assigned the instance to be a property on itself. Finally, we deleted the instance.

By deleting the instance, it's no longer accessible in our Python program. However, Python didn't destroy the instance from memory. The instance doesn't have a reference count of zero because it has a reference to itself.

We call this type of problem a reference cycle, and you can't solve it by reference counting. This is the point of the generational garbage collector, which is accessible by the gc module in the standard library.

Generational garbage collector terminology

There are two key concepts to understand with the generational garbage collector. The first concept is that of a generation.

The garbage collector is keeping track of all objects in memory. A new object starts its life in the first generation of the garbage collector. If Python executes a garbage collection process on a generation and an object survives, it moves up into a second, older generation. The Python garbage collector has three generations in total, and an object moves into an older generation whenever it survives a garbage collection process on its current generation.

The second key concept is the threshold. For each generation, the garbage collector module has a threshold number of objects. If the number of objects exceeds that threshold, the garbage collector will trigger a collection process. For any objects that survive that process, they're moved into an older generation.

Unlike the reference counting mechanism, you may change the behavior of the generational garbage collector in your Python program. This includes changing the thresholds for triggering a garbage collection process in your code, manually triggering a garbage collection process, or disabling the garbage collection process altogether.

Let's see how you can use the gc module to check garbage collection statistics or change the behavior of the garbage collector.

Using the GC module

In your terminal, enter python to drop into a Python REPL.

Import the gc module into your session. You can then check the configured thresholds of your garbage collector with the get_threshold() method:

```
>>> import gc
>>> gc.get_threshold()
(700, 10, 10)
```

By default, Python has a threshold of 700 for the youngest generation and 10 for each of the two older generations.

You can check the number of objects in each of your generations with the get_count() method:

```
>>> import gc
>>> gc.get_count()
(596, 2, 1)
```

In this example, we have 596 objects in our youngest generation, two objects in the next generation, and one object in the oldest generation.

As you can see, Python creates a number of objects by default before you even start executing your program. You can trigger a manual garbage collection process by using the gc.collect() method:

```
>>> gc.get_count()
(595, 2, 1)
>>> gc.collect()
57
>>> gc.get_count()
(18, 0, 0)
```

Running a garbage collection process cleans up a huge amount of objects—577 in the first generation and three more in the older generations.

You can alter the thresholds for triggering garbage collection by using the set_threshold() method in the gc module:

```
>>> import gc
>>> gc.get_threshold()
(700, 10, 10)
>>> gc.set_threshold(1000, 15, 15)
>>> gc.get_threshold()
(1000, 15, 15)
```

In the example above, we increase each of our thresholds from their defaults. Increasing the threshold will reduce the frequency at which the garbage collector runs. This will be less computationally expensive in your program at the expense of keeping dead objects around longer.

Now that you know how both reference counting and the garbage collector module work, let's discuss how you should use this when writing Python applications.

What does Python's garbage collector mean for you as a developer

We've spent a fair bit of time discussing memory management generally and its implementation in Python. Now it's time to make it useful—how should you use this information as a developer of Python programs?

General rule: Don't change garbage collector behavior

As a general rule, you probably shouldn't think about Python's garbage collection too much. One of the key benefits of Python is how it enables developer productivity. Part of the reason for this is because it's a high-level language that handles a number of low-level details for the developer.

Manual memory management is more relevant for constrained environments. If you do find yourself with performance limitations that you think may be related to Python's garbage collection mechanisms, it will probably be more useful to increase the power of your execution environment rather than to manually alter the garbage collection process. In a world of Moore's law, cloud computing, and cheap memory, more power is readily accessible.

This is even truer given that Python generally doesn't release memory back to the underlying operating system. Any manual garbage collection process you do to free memory may not give you the results you want. For more details in this area, refer to this post on memory management in Python.

Disabling the garbage collector

With that caveat aside, there are situations where you may want to manage the garbage collection process. Remember that reference counting, the main garbage collection mechanism in Python, can't be disabled. The only garbage collection behavior you can alter is the generational garbage collector in the gc module.

One of the more interesting examples of altering the generational garbage collector came from Instagram disabling the garbage collector altogether.

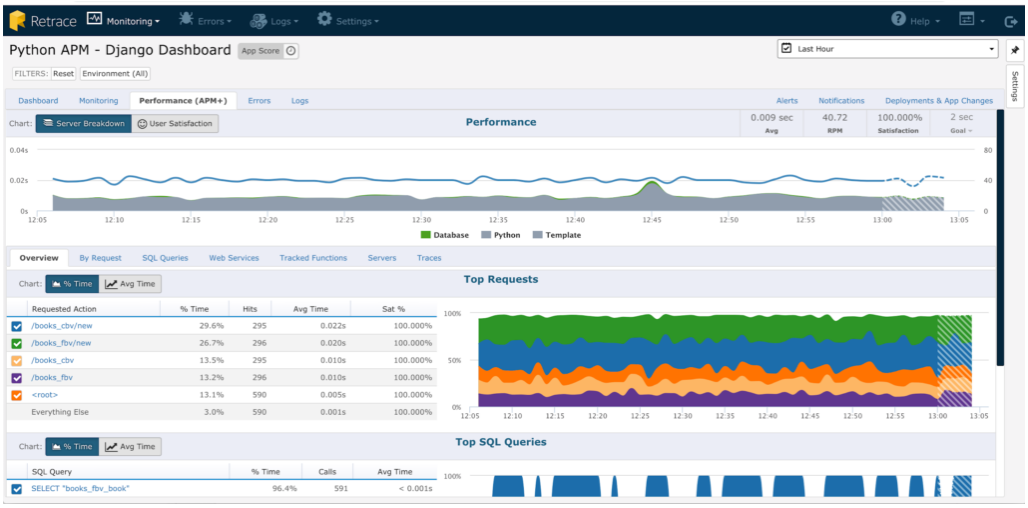
Instagram uses Django, the popular Python web framework, for its web applications. It runs multiple instances of its web application on a single compute instance. These instances are run using a master-child mechanism where the child processes share memory with the master.

The Instagram dev team noticed that the shared memory would drop sharply soon after a child process spawned. When digging further, they saw that the garbage collector was to blame.

The Instagram team disabled by the garbage collector module by setting the thresholds for all generations to zero. This change led to their web applications running 10% more efficiently.

While this example is interesting, make sure you're in a similar situation before following the same path. Instagram is a web-scale application serving many millions of users. To them, it's worth it to use some non-standard behavior to squeeze every inch of performance from their web applications. For most developers, Python's standard behavior around garbage collection is sufficient.

If you think you may want to manually manage garbage collection in Python, make sure you understand the problem first. Use tools like Stackify's Retrace to measure your application performance and pinpoint issues. Once you fully understand the problem, then take steps to fix it.



Wrapping up

In this post, we learned about garbage collection in Python. We started by covering the basics of memory management and the creation of automatic memory management. We then looked at how garbage collection is implemented in Python, through both automatic reference counting and a generational garbage collector. Finally, we reviewed how this matters to you as a Python developer.

While Python handles most of the hard parts of memory management for you, it's still helpful to know what's happening under the hood. From reading this post, you now know that you should avoid reference cycles in Python, and you should know where to look if you need greater control over Python's garbage collector.